Calcul de requêtes dans une base de données

Nathan TILLIER

Sommaire

1. Requêtes conjonctives
2. Algorithme de Yannakakis
3. Résultats
4. Conclusion

Requêtes conjonctives

Base de données

Une base de données est un ensemble de relations (tables). Le coût majeur des requêtes provient de l'opération de **jointure**.

```
CREATE TABLE commune (
    INT id PRIMARY KEY,
    INT departement,
    CHAR nom
)
```

Identifiant	Département	Nom
1	75	Paris
2	29	Brest
3	69	Lyon

Jointure et approche naïve

On fait un produit cartésien. **Problème :** Explosion combinatoire de la taille intermédiaire.

commune.id	commune.nom	commune.departement	departement.id	departement.nom
1	Paris	75	75	Île-de-France
			29	Finistère
			80	Somme
2	Brest	29	75	Île-de-France
			29	Finistère
			80	Somme
3	Lyon	69	75	Île-de-France
			29	Finistère
			80	Somme

© Calcul de requêtes dans une base de données

Requête conjonctive

$$q(\text{c.nom}, \text{d.nom}) : -\text{commune}(\text{c.id}, \text{c.nom}, \text{d.id}), \text{dep}(\text{d.id}, \text{d.nom})$$

Requête conjonctive

```sql
SELECT c.nom, d.nom
FROM commune AS c
JOIN departement AS d ON c.departement = d.id
```

Requête non conjonctive

```sql
SELECT c.nom, d.nom
FROM commune AS c
JOIN departement AS d ON c.departement = d.id
WHERE c.nom LIKE "Le %"
```

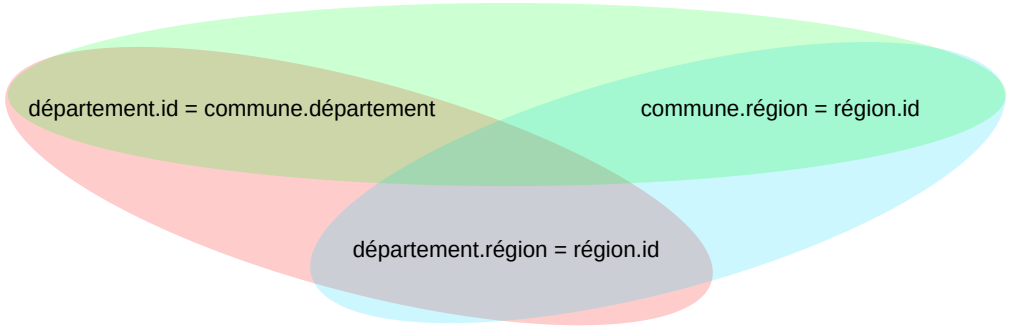
Requête conjonctive acyclique

Requête conjonctive acyclique

```sql
SELECT c.nom, d.nom, r.nom
FROM commune AS c
JOIN departement AS d ON c.departement = d.id
JOIN region AS r ON d.region = r.id
```

Requête conjonctive cyclique

```sql
SELECT c.nom, d.nom, r.nom
FROM commune AS c
JOIN departement AS d ON c.departement = d.id
JOIN region AS r ON d.region = r.id AND c.region = r.id
```



Algorithme de Yannakakis

Arbre de jointure

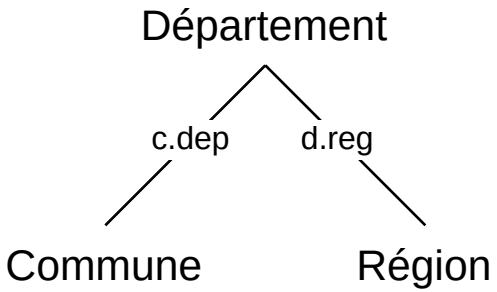
Définition : arbre de jointure d'une requête

Graphe T vérifiant

- les sommets de T sont les relations de la requête
- le sous-graphe induit par les relations contenant une même variable partagée est connexe
- T est un arbre

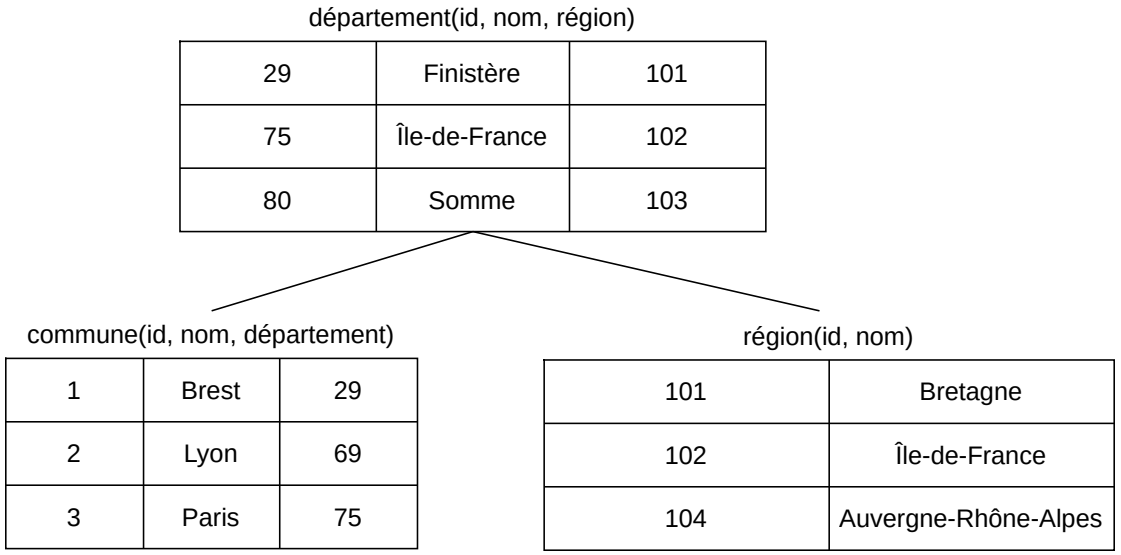
Exemple

```
SELECT c.nom, d.nom, r.nom
FROM commune AS c
JOIN departement AS d ON c.departement = d.id
JOIN region AS r ON d.region = r.id
```



Algorithme de Yannakakis

- **Entrée :** une requête conjonctive q, une instance I
- **Sortie :** $q(I)$
- $T \leftarrow$ arbre de jointure de q avec une racine
- $J \leftarrow$ instance réduite issue de I
- $R_1, \ldots, R_n \leftarrow$ parcours postfixe de T
- **Pour** $i = 1, \ldots, n$ **faire**
  - $S_1, \ldots, S_k$ enfants de R_i dans T
  - **Pour** $j = 1, \ldots, k$ **faire**
    - $R_i^J \leftarrow \pi_{\mathrm{vars}(R_i) \cup \mathrm{head}(q)}(R_i^J \bowtie S_j^J)$
  - **Fin**
- **Fin**



Résultats

Jeu de données

Régions, département et communes françaises.

```
CREATE TABLE communes (
    INT id PRIMARY KEY,
    INT region,
    INT departement,
    CHAR nom
)

CREATE TABLE departements (
    INT id PRIMARY KEY,
    INT region,
    CHAR nom
)

CREATE TABLE regions (
    INT id PRIMARY KEY,
    INT cheflieu,
    CHAR nom
)
```

Communes

Identifiant	Région	Département	Nom
60057	32	60	Beauvais

Départements

Identifiant	Région	Nom
32	11	Paris

Régions

Identifiant	Cheflieu	Nom
60	59350	Oise

Exécution des requêtes

```python
class Table:
    name: str
    rows: List[tuple]

    def import_from(self, csv_path: str)
    def export_to(self, csv_path: str)
```

```python
class Instance:
    tables: dict[str, Table]

    def add_table(self, table: Table)
    def bottom_up(self, query: Query, node: JoinNode)
    def query(self, Query)
```

```python
communes = Table("commune")
communes.import_from("code/data/v_commune_2025.csv")
```

```python
Query(
    Head("requête", c_nom, d_nom, r_nom),
    Atom("commune", _, c_nom, d_id),
    Atom("département", d_id, d_nom, d_region),
    Atom("région", d_region, r_nom),
)
```

Exemples de requête

Commune, département, région

```
SELECT commune.nom, departement.nom, region.nom FROM commune
JOIN departement ON departement.numero = commune.departement
JOIN region ON region.numero = commune.region
```

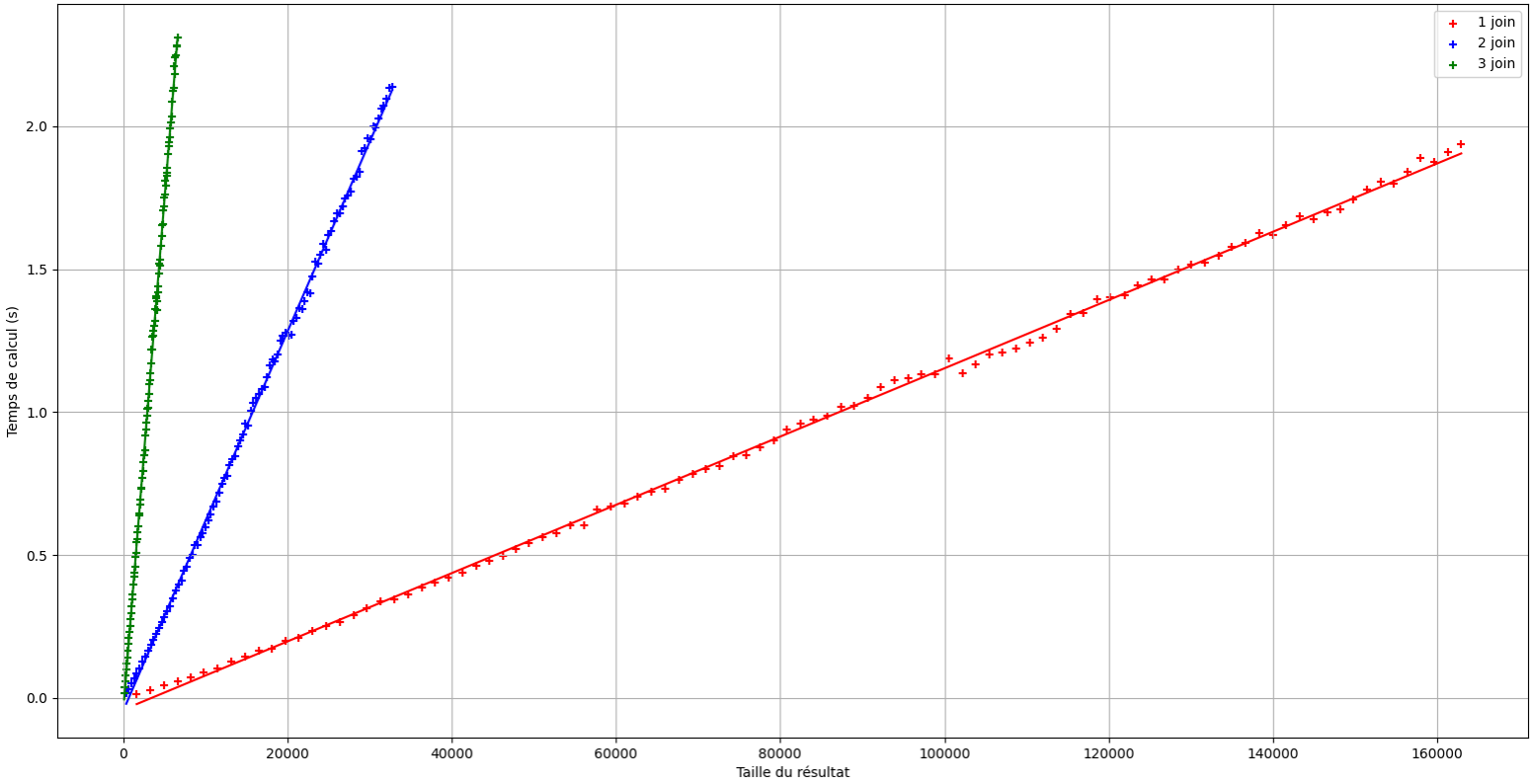
- Version simple : 0.17 secondes
- Version double parcours : 0.08 secondes

Commune, cheflieu

```
SELECT commune.nom, departement.nom, cheflieu.nom FROM commune
JOIN departement ON departement.numero = commune.departement
JOIN commune as cheflieu ON commune.numero = departement.cheflieu
```

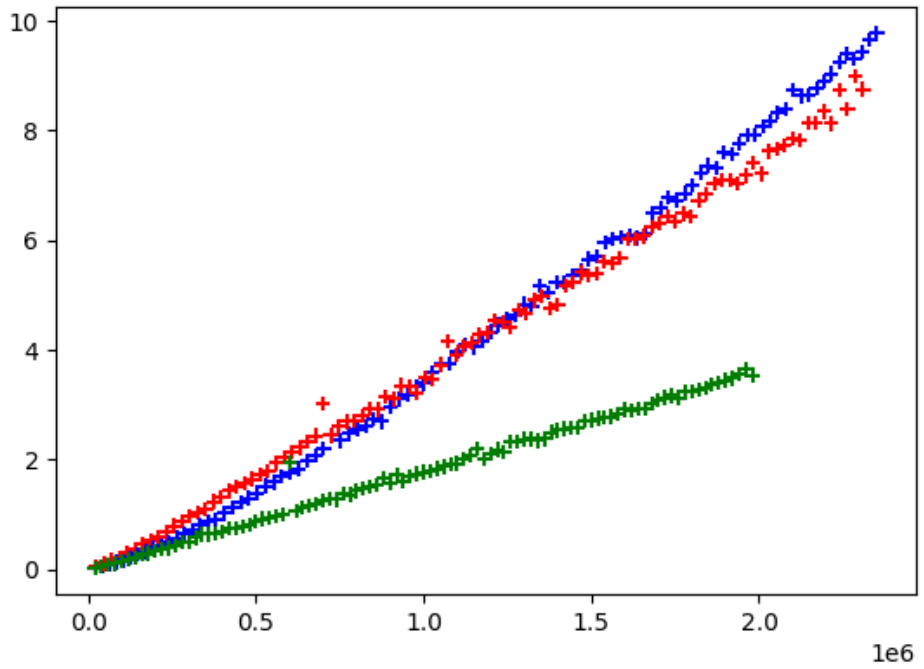
- Version simple : 0.09 secondes
- Version double parcours : 0.06 secondes

Statistiques

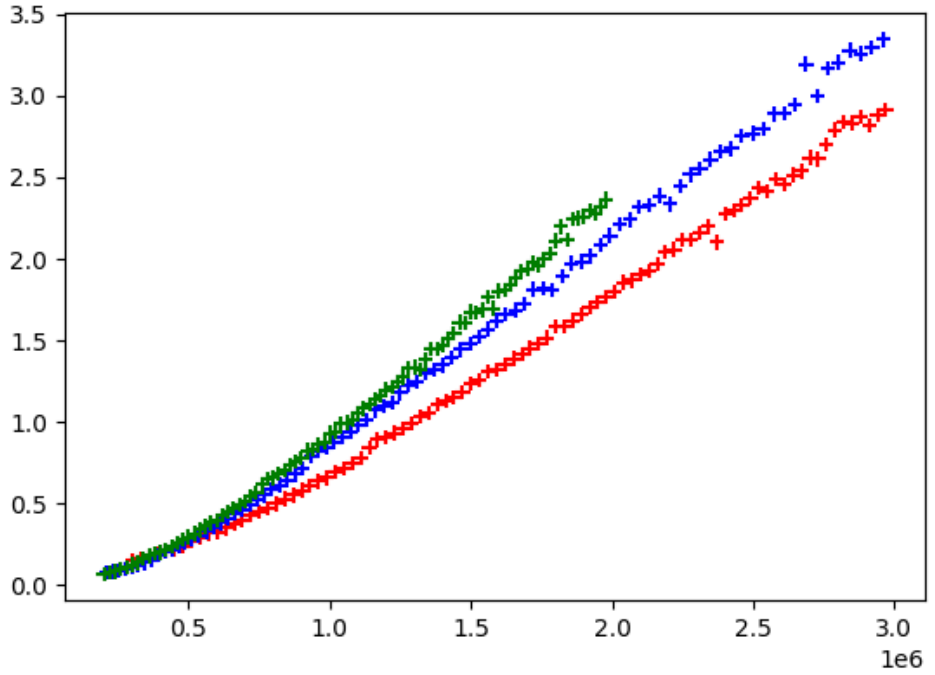


Statistiques

Temps d'exécution de la requête (seconde) selon la taille de la base de donnée.



`Implémentation Python`



`sqlite3 (inmemory)`

Conclusion

Conclusion

Efficacité

- Une implémentation en C améliorerait les performances et la gestion de la mémoire.
- Il est possible de paralléliser l'algorithme, avec un fil réalisant chaque fils de la racine.
- La constante est assez élevée, dans la pratique ce n'est pas l'algorithme qui est utilisé.

Requêtes

- On peut étendre la structure de requête pour ajouter des conditions sur les valeurs lors du premier passage (bottom-up).

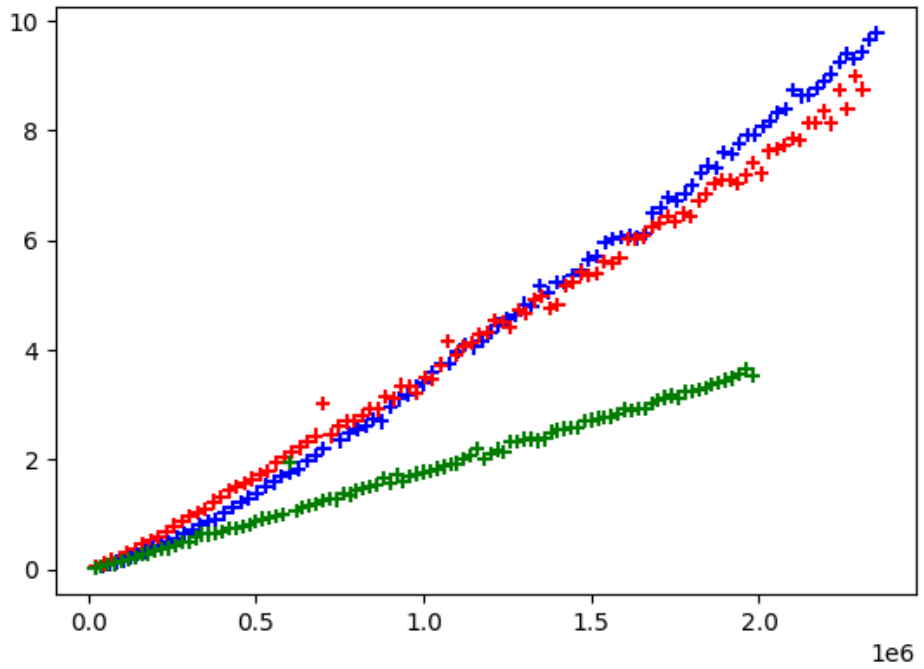
Annexe

Sommaire de l'annexe

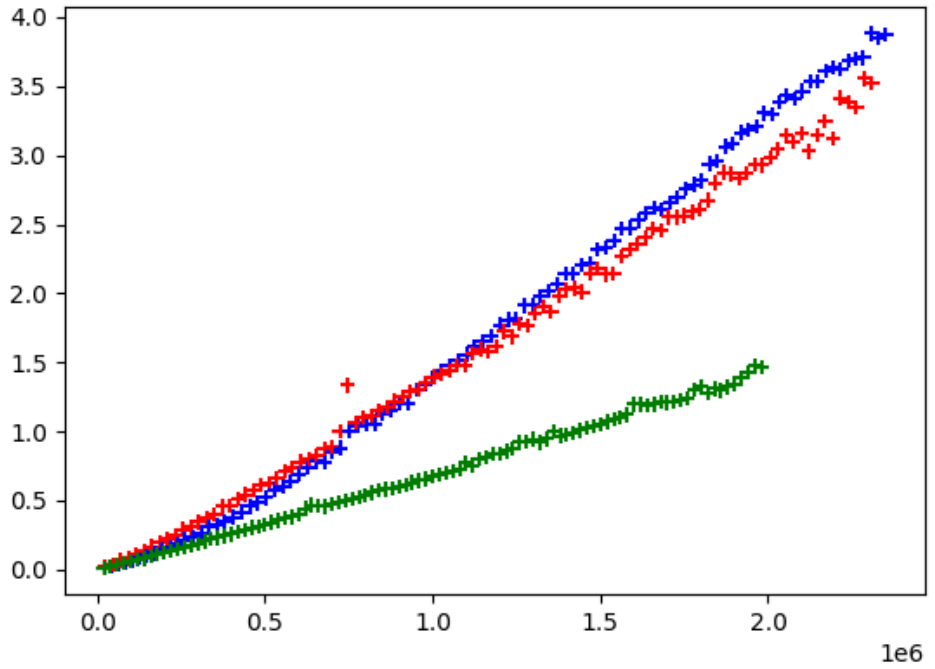
- Statistiques selon le mode de puissance de l'ordinateur
- Statistiques avec la réécriture en C
- Statistiques avec plus de résultats finaux
- Hypergraphe d'une requête
- Arbre de jointure d'une requête
- Réduction GYO et construction de l'arbre de jointure
- Éléments de preuve (Yannakakis et GYO)
- NP-complétude du calcul de requêtes

Statistiques (bis)

Temps d'exécution de la requête (seconde) selon la taille de la base de donnée et du résultat.

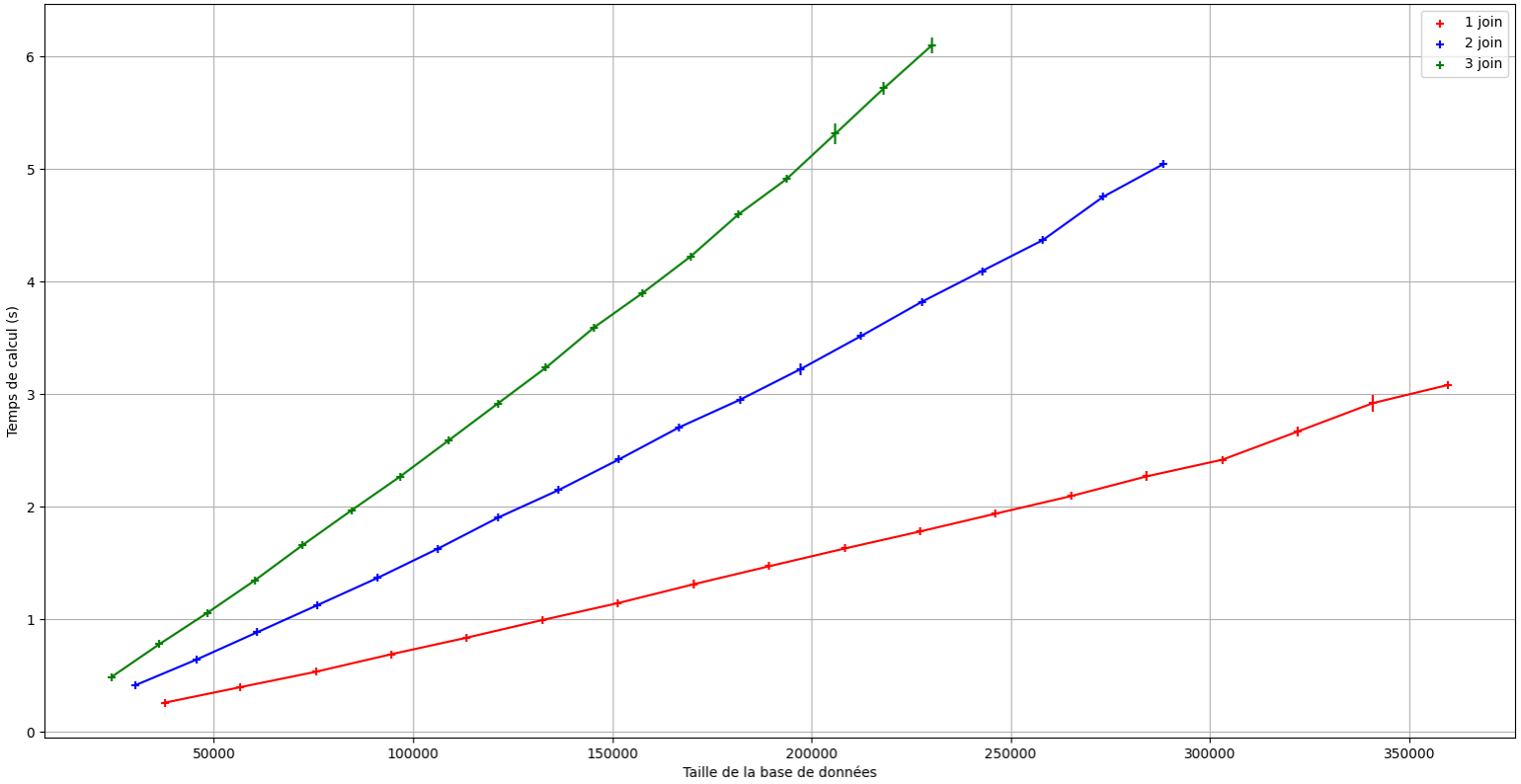


Mode "économie d'énergie"



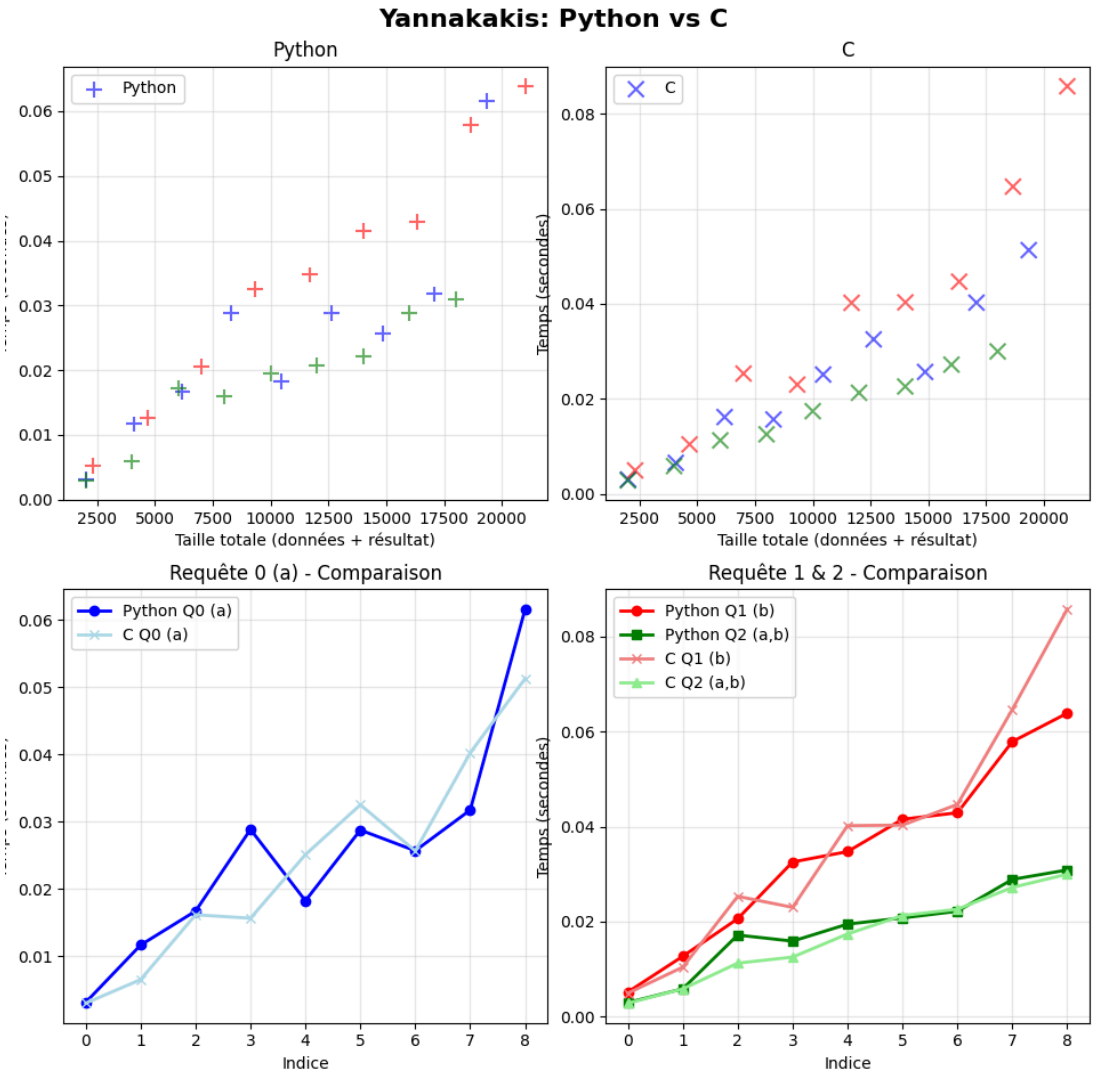
Mode "performance"

Statistiques



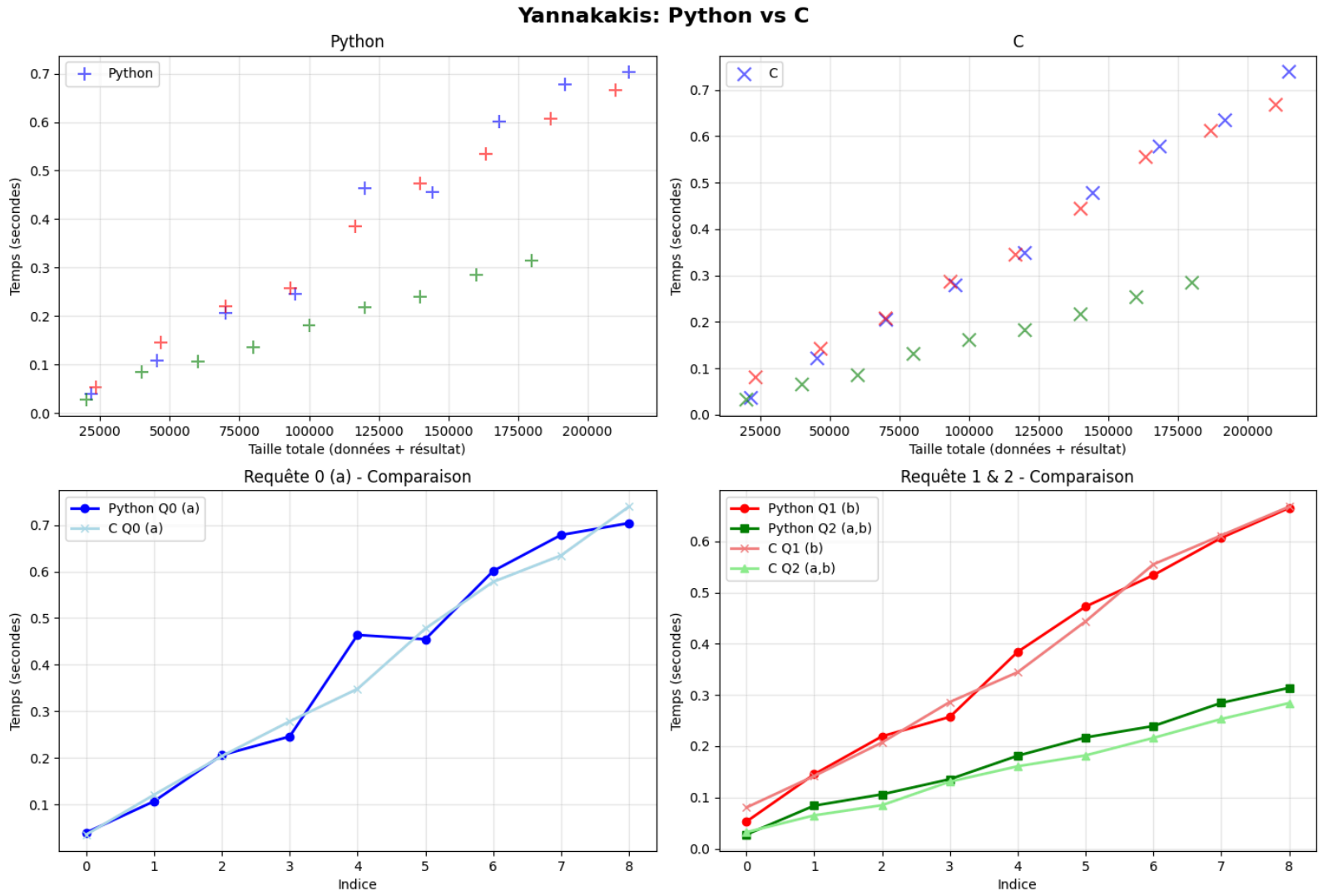
Courbes avec beaucoup plus de résultats finaux (d = 0.8)

Python versus C



© Calcul de requêtes dans une base de données

Python versus C



© Calcul de requêtes dans une base de données

Hypergraphe d'une requête

Définition

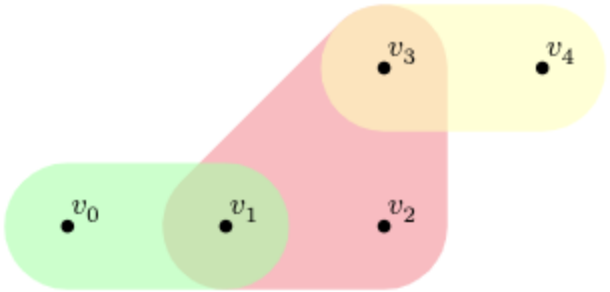
$$q(x_1, \ldots, x_n) :- R_1(\vec{y_1}), \ldots, R_p(\vec{y_p})$$

- $V = \{x_1, \ldots, x_n\}$
- les arêtes sont les variables apparaissant dans chacune des clauses atomiques

$$E_i = V \cap \mathrm{var}(\vec{y_i})$$

Exemple

$$q(v_0, v_1, v_2, v_3, v_4) :- R_{\mathrm{vert}}(v_0, v_1), R_{\mathrm{rouge}}(v_1, v_2, v_3), R_{\mathrm{jaune}}(v_3, v_4)$$



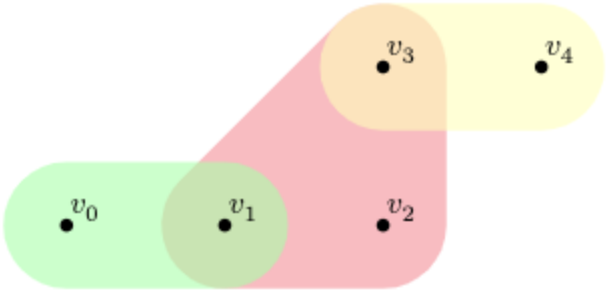
Arbres de jointure

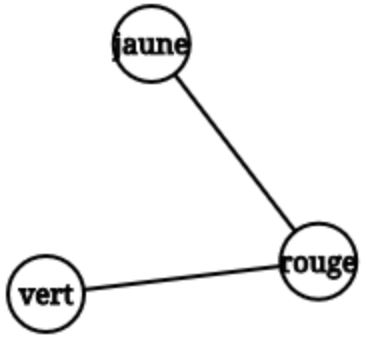
Définition

Soit H un hypergraphe sur X, et T un graphe ayant pour sommet les arêtes de H.
T est un arbre de jointure si

- pour tout sommet v de H, le sous-graphe de T engendré par l'ensemble des sommets de T contenant v est connexe
- T est un arbre

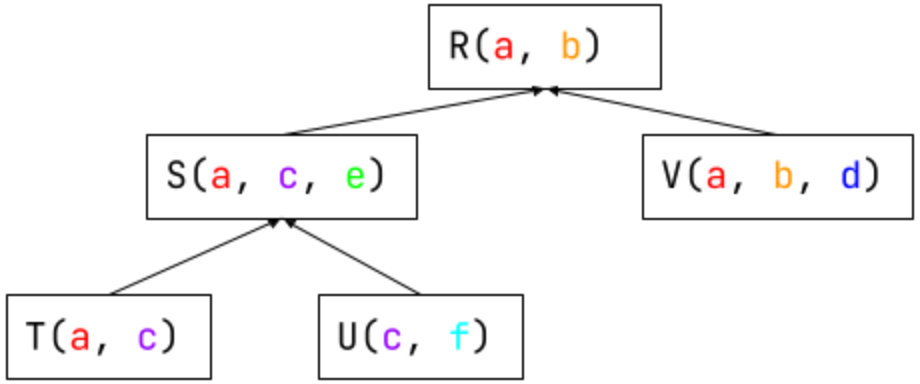
Un tel hypergraphe est dit α-acyclique.





Arbres de jointure

```
q(a, b, c, d, e, f) :- R(a, b), S(a, c, e), T(a, c), U(c, f), V(a, b, d)
```



Réduction Graham-Yu-Ozsoyoglu

Pour déterminer si un hypergraphe est α-acyclique.

Entrée : hypergraphe H
Sortie : booléen indiquant si H est acyclique

- **Tant que** H a une oreille **faire**
  - e oreille de H
  - retirer tous les sommets exclusivement dans e
  - retirer e
- **Fin**
- **Renvoyer** H est l'hypergraphe vide

```python
def est_acyclique(self):
    h = self.copier()
    restant = set(range(h.ordre))
    sommets_compte = [h.decompte(s) for s in range(h.ordre)]

    while True:
        sortir = True

        # On enlève tous les sommets isolés
        for s in restant:
            if sommets_compte[s] <= 1:
                restant.discard(s)
                h.retirer_sommet(s)
                sortir = False

        # On enlève toutes les arêtes inclues dans d'autre
        for indice, a in enumerate(h.aretes):
            for b in h.aretes:
                if a.dans(b) and a.indice != b.indice:
                    for s in a.sommets:
                        sommets_compte[s] -= 1
                    h.enlever_arete(indice)
                    sortir = False
                    break

        # On continue éventuellement l'algorithme
        if sortir or len(restant) == 0:
            break

    return len(restant) == 0
```

Construction de l'arbre de jointure

Il suffit de reprendre l'algorithme précédent, en rajoutant des arêtes dans l'arbre au fur et à mesure que des sommets de l'hypergraphe sont retirés.

```
def est_acyclique(self):
    arbre = [[] for _ in range(self.nb_aretes)]

    # [...]

    while True:
        # [...]

        # On enlève toutes les arêtes inclues
        for indice, a in enumerate(h.aretes):
            for b in h.aretes:
                if a.dans(b) and a.indice != b.indice:
                    arbre[a.indice].append(b.indice)
                    arbre[b.indice].append(a.indice)
                    # [...]

        # [...]

    return arbre if len(restant) == 0 else None
```

Réduction GYO

Éléments de preuve (correction)

- dès qu'on retire un sommet isolé, on ne perd aucune information sur T arbre de jointure. On retire d'ailleurs son arête avec si elle ne contient que lui.
- si une arête E est incluse dans une autre arête F, alors les sommets E et F sont nécessairement reliés dans T (on crée la connexité sans briser l'acyclicité).
- si une de ces étapes n'est pas faisable et que l'hypergraphe n'est pas vide, on ne peut pas contruire T : on créera forcément un cycle.

Éléments de preuve (complexité)

- Enlever les sommets isolés : $O(|X|)$
- Enlever les arêtes : $O(|H|^2|X|)$
- Boucle *while*: au plus $|X| + |H|$ itérations (on retire à chaque fois au moins un sommet ou une arête)

Algorithme de Yannakakis

Éléments de preuve (correction)

- Lors des projections successives on ne perd aucune information (locale et globale) nécessaire à la construction du résultat souhaité.

Éléments de preuve (complexité)

- Afin d'obtenir la complexité souhaitée, on montre qu'aucun résultat intermédiaire ne peut être plus grand qu'un polynôme en la taille de l'entrée et de la sortie. On considère un noeud R_i avec pour enfants $S_1, \ldots, S_n$ dans un arbre de jointure correspondant à une requête.
- Cet noeud R_i (noeud dans l'arbre de jointure) sera remplacé par un noeud $R_i^J = \pi_{Y \cup Z}(T)$ où $Y = \mathrm{vars}(R_i)$ et $Z = \mathrm{head}(q) \setminus Y$ et $T = R_i \bowtie S_1 \bowtie \ldots \bowtie S_n$
- Puisque Y et Z sont disjoints, $\pi_{Y \sqcup Z}(T) \subseteq \pi_Y(T) \times \pi_Z(T)$. On remarque que $\pi_Y(T)$ est de taille bornée par celle de R_i, et que $\pi_Z(T)$ l'est par la taille de la sortie (car l'exploration se fait dans l'ordre postfixe, donc on a déjà réduit les *dangling tuples*).

NP-complétude (calcul de requêtes)

Éléments de preuve (dans NP)

- Certificat : valeurs prises par chaque variables de la requête (de l'en-tête et du corps), polynomial en la taille de celle-ci. Homomorphisme.
- Vérification : la relation est-elle dans la base de données ? Linéaire en la taille de la base de données.

NP-complétude (calcul de requêtes)

Éléments de preuve (NP-difficile)

- On construit une instance et une requête permettant de réduire le problème choisi.
- 3-SAT :
  - L'instance contient toutes les clauses positives (un terme à 1), et les négations pour les variables temporaires : $\neg x_i \equiv x'_i$ et $x_i \vee x'_i$
  - $I = \{Clause(0,0,1), \ldots, Clause(1,1,1)\} \cup \{Negation(0,1), Negation(1,0)\}$
  - Requête : $q(x_1, \ldots, x_k) : -\{Clause(x_i, x_j, x_k)\}, \{Negation(x_i, x'_i)\}$
- k-clique : avec $G = (S, A)$
  - Instance : $I = \{Noeud(s) | s \in S\} \cup \{Arete(s,t); \{s,t\} \in A\}$
  - Requête :
    $q(x_1, \ldots, x_k) : -Noeud(x_1), \ldots, Noeud(x_k), \{Arete(x_i, x_j); i \neq j \in [1,k]\}$

Sources

- CODD E. F. : A relational model of data for large shared data banks. : https://doi.org/10.1145/362384.362685
- KOUTRIS P. : Foundations of Data Management : Acyclic Conjunctive Queries : https://pages.cs.wisc.edu/~paris/cs784-f19/lectures/lecture2.pdf
- YANNAKAKIS M. : Algorithms for Acyclic Database Schemes : Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7, (1981)
- BERGE C. : Graphes et hypergraphes : Graphes et hypergraphes Dunod (1970)
- HU X. : Algorithmic Aspects of Query Processing - Traditional Query Processing : https://cs.uwaterloo.ca/~xiaohu/courses/CS848/CS848-F25-03-Yannakakis-part2.pdf
- LUO Z., BROECK W. V. DEN, BROECK G. V. DEN, WANG Y. R. : Algorithms for Optimizing Acyclic Queries : https://arxiv.org/abs/2509.14144
- WOLFGANG GATTERBAUER : Toward Responsive DBMS : Part 3 Acyclic Queries & Enumeration : https://www.youtube.com/watch?v=toi7ysuyRkw